1.
Write a program that uses functions to perform the following operations on singly linked list: i. Creation ii. Insertion iii. Deletion iv. Traversal

```
include <stdio.h>

#include <stdlib.h>

*Defination of node*

struct node {

int data;

strict node *next;
};

Strict node *head = NULL

*function prototypes*

void createe();

void Insert();

void deletee();

void traverse();

*Main function*

int main() {

int choice;

while(1) {

printf("\n..... Singly Linked List Menu…\n");

printf("1. create \n");

printf(" 2. Insert \n");
```

```
printf(" 3. delete \n");

printf(" 4. Traverse in");
printf("5.Exit\n");

printf ("enter your choice ");

scanf("%d", & choice);
switch (choice)
{
case 1 :create();

break;

case 2:insert();

break;

case 3: deletee();

break;

case 4: traverse();

break;

case 5: exit(o);

default : printf("Invalid choice");

}
}
return 0;

}


/*create Linked list*/

void create(){

int n, i, value;

struct node* temp,* newnde;
```

```
printf("Enter number of nodes:");

Scanf("%d",&n);

for (i=0; i<n;i++){

new node = (Struct node *)malloc

Printf ("Enter data:");

scanf("%d",& value);
new node ->data = value;

new node-> next = NULL;

if (head =NULL);

{

head =newnode
}
else

{
temp=head;

while (temp->next!= NULL)

temp= temp->next;

temp-> next= newnode;
}
}
}
/* Insert at End */
void insert() {
   int value;
   struct node *newnode, *temp;

   newnode = (struct node *)malloc(sizeof(struct node));
   printf("Enter value to insert: ");
   scanf("%d", &value);
```

```
    newnode->data = value;
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newnode;
    }
}

/* Delete a Node */
void deletee() {
    int value;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &value);

    temp = head;

    if (temp->data == value) {
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }

    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Value not found\n");
    } else {
        prev->next = temp->next;
```

```
        free(temp);
        printf("Node deleted\n");
    }
}
/* Traverse the List */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");}
```

6:05

0.20 KB/S 35%

Coding C

Auto saved at 18:05:15

RUN MENU

```
1 #include <stdio.h>
2
3 #
4
5 *
6
7 s
8
9 i
10
11 s
12 }
13
14 S
15
16 *
17
18 v
19
20 v
21
22 v
23
```

Ooop...

```
default.c:5:2: error: type specifier missing,
defaults to 'int'; ISO C99 and later do not
support implicit int [-Wimplicit-int]
5 | *Defination of node*
  | ^
  | int
default.c:5:12: error: expected ';' after top
level declarator
5 | *Defination of node*
  |          ^
  |          ;
default.c:14:1: error: unknown type name
'Strict'; did you mean 'struct'?
14 | Strict node *head = NULL
   | ^~~~~~
   | struct
default.c:16:2: error: use of undeclared
identifier 'function'
16 | *function prototypes*
```

Write a program that uses functions to perform the following operations on doubly linked list: i. Creation ii. Insertion iii. Deletion iv. Traversal

Duoble linked list

```
include <stdio.h>
#include <stdlib.h>
struct node {
   int data;
   struct node *prev;
   struct node *next;
};
struct node *head = NULL;
void create();
void insert();
void deletee();
void traverse();
void display();
int main() {
   int choice;
   while (1) {
      printf("\n--- Doubly Linked List Menu ---\n");
      printf("1. Create\n");
      printf("2. Insert\n");
      printf("3. Deletee\n");
      printf("4. Traverse\n");
      printf("5.Display\n");
      printf("6. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);
      switch (choice) {
         case 1: create();
               break;
         case 2: insert();
               break;
         case 3: deletee();
               break;
         case 4: traverse();
               break;
         case 5: display();
               break;
         case 6: exit(0);
```

```
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}
void create() {
    int n, i, value;
    struct node *temp, *newnode;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &value);
        newnode->data = value;
        newnode->prev = NULL;
        newnode->next = NULL;
        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
                temp->next = newnode;
                newnode->prev = temp;
        }
    }
}
void insert() {
    int value;
    struct node *newnode, *temp;
    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &value);
    newnode->data = value;
    newnode->prev = NULL;
    newnode->next = NULL;
    if (head == NULL) {
        head = newnode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
            temp->next = newnode;
```

```
            newnode->prev = temp;
    }
}
void deletee() {
    int value;
    struct node *temp;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Enter value to delete: ");
    scanf("%d", &value);
    temp = head;
    if (temp->data == value) {
        head = temp->next;
        if (head != NULL)
            head->prev = NULL;
        free(temp);
        printf("Node deleted\n");
        return;
    }
    while (temp != NULL && temp->data != value)
        temp = temp->next;
    if (temp == NULL) {
        printf("Value not found\n");
    } else {
        temp->prev->next = temp->next;
        if (temp->next != NULL)
            temp->next->prev = temp->prev;
        free(temp);
        printf("Node deleted\n");
    }
}
void traverse() {
    struct node *temp;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Forward Traversal: ");
    temp = head;
    while (temp->next != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
```

```
    }
    printf("%d\n", temp->data);
    printf("Backward Traversal: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}
void display()
{
    struct node*temp;
    if(head==NULL)
    {
        printf("\n list is empty");
        return;
    }
    else
    {
        temp=head;
        while(temp->next!=head)
        {
            printf("%d",temp->data);
            temp=temp->next;
        }
    }
}
```

Write a program that uses functions to perform the following operations on circular linked i. Creation ii. Insertion iii. Deletion iv. Traversal

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

/* Function declarations */
void create();
void insert();
void delete();
```

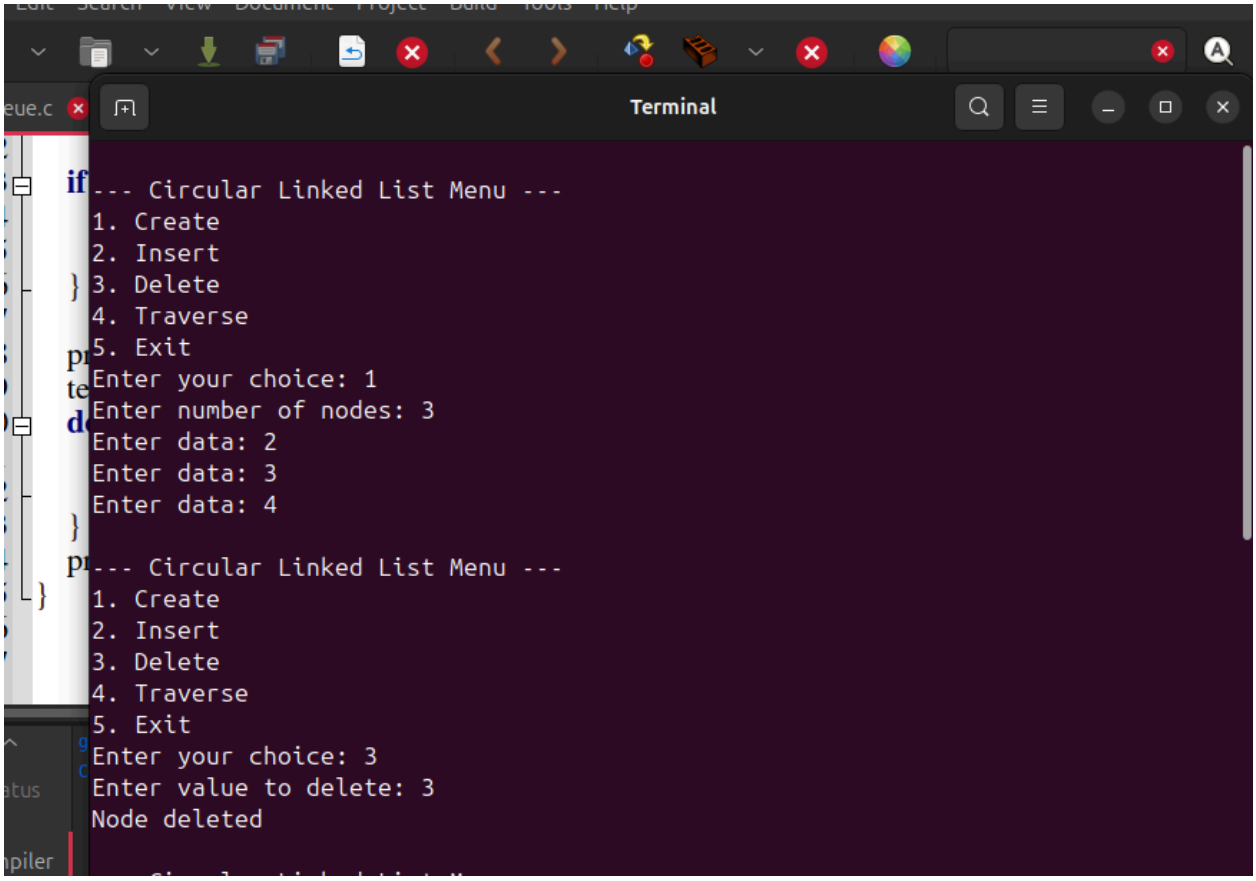
```
void traverse();

/* Main function */
int main() {
    int choice;

    while (1) {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Create\n");
        printf("2. Insert\n");
        printf("3. Delete\n");
        printf("4. Traverse\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create(); break;
            case 2: insert(); break;
            case 3: delete(); break;
            case 4: traverse(); break;
            case 5: exit(0);
            default: printf("Invalid choice\n");
        }
    }
    return 0;
}

/* Create circular linked list */
void create() {
    int n, i, val;
    struct node *newnode, *temp;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &val);

        newnode->data = val;

        if (head == NULL) {
```

```
            head = newnode;
            newnode->next = head;
        } else {
            temp = head;
            while (temp->next != head)
                temp = temp->next;

            temp->next = newnode;
            newnode->next = head;
        }
    }
}

/* Insert at end */
void insert() {
    int val;
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    printf("Enter value to insert: ");
    scanf("%d", &val);

    newnode->data = val;

    if (head == NULL) {
        head = newnode;
        newnode->next = head;
    } else {
        temp = head;
        while (temp->next != head)
            temp = temp->next;

        temp->next = newnode;
        newnode->next = head;
    }
}

/* Delete node by value */
void delete() {
    int val;
    struct node *temp, *prev;

    if (head == NULL) {
        printf("List is empty\n");
```

```
        return;
    }

    printf("Enter value to delete: ");
    scanf("%d", &val);

    temp = head;
    prev = NULL;

    /* If head node is to be deleted */
    if (temp->data == val) {
        if (temp->next == head) {
            free(temp);
            head = NULL;
            printf("Node deleted\n");
            return;
        }
        prev = head;
        while (prev->next != head)
            prev = prev->next;

        prev->next = temp->next;
        head = temp->next;
        free(temp);
        printf("Node deleted\n");
        return;
    }

    prev = temp;
    temp = temp->next;

    while (temp != head && temp->data != val) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == head) {
        printf("Value not found\n");
    } else {
        prev->next = temp->next;
        free(temp);
        printf("Node deleted\n");
    }
}
```

```
/* Traverse circular linked list */
void traverse() {
    struct node *temp;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Circular Linked List: ");
    temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(back to head)\n");
}
```

```
Node deleted

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 4
Circular Linked List: 2 -> 4 -> (back to head)

--- Circular Linked List Menu ---
1. Create
2. Insert
3. Delete
4. Traverse
5. Exit
Enter your choice: 5


------------------
(program exited with code: 0)
Press return to continue
```



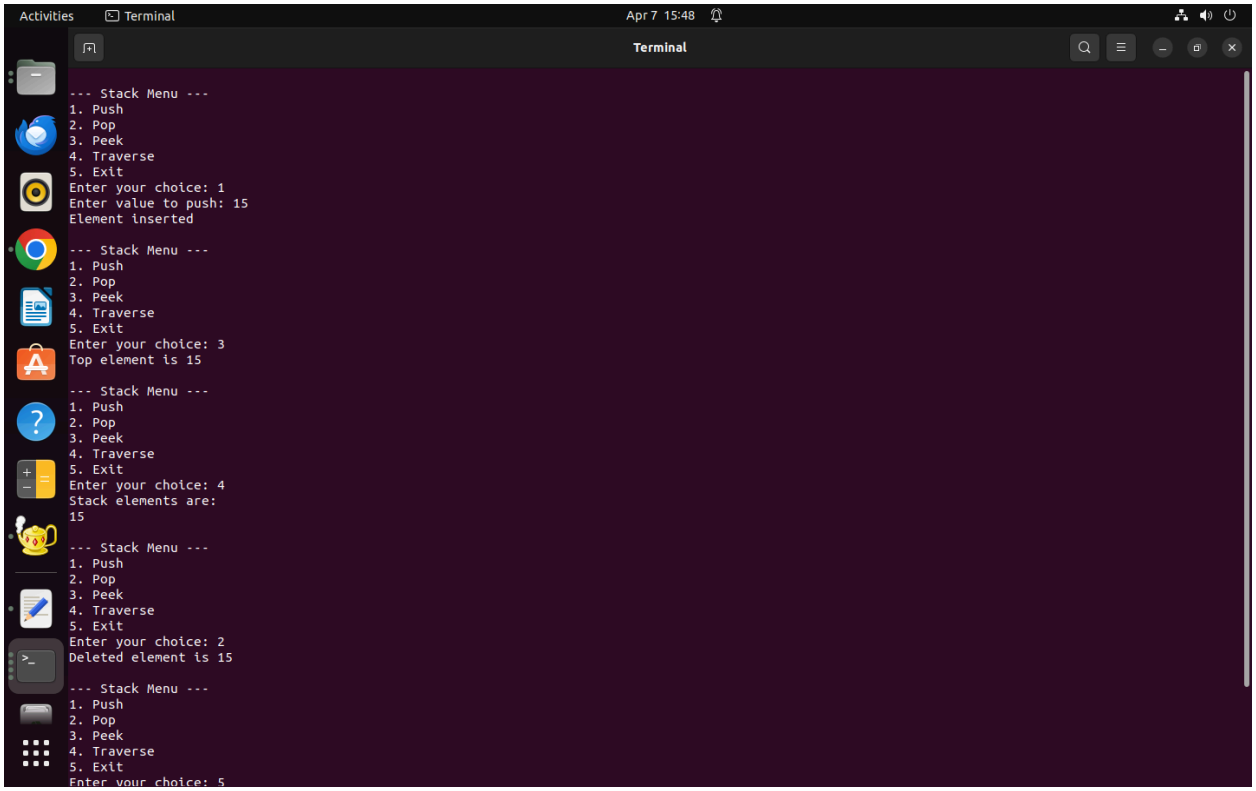
4. Write a program that implement stack (its operations) using
 i. Arrays ii. Linked List

i. Arrays

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int stack[MAX];
int top = -1;

/* Function declarations */
void push();
void pop();
void peek();
void display();

int main()
```

```
{
        int choice;

        while(1)
        {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
        case 1: push(); break;
        case 2: pop(); break;
        case 3: peek(); break;
        case 4: display(); break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
        }
        }
}

/* Push operation */
void push()
{
        int item;

        if(top == MAX - 1)
        {
        printf("Stack Overflow\n");
        return;
        }

        printf("Enter element to push: ");
        scanf("%d", &item);

        top++;
        stack[top] = item;

        printf("Element pushed successfully\n");
```

```
}

/* Pop operation */
void pop()
{
	if(top == -1)
	{
	printf("Stack Underflow\n");
	return;
	}

	printf("Popped element: %d\n", stack[top]);
	top--;
}

/* Peek operation */
void peek()
{
	if(top == -1)
	{
	printf("Stack is empty\n");
	return;
	}

	printf("Top element: %d\n", stack[top]);
}

/* Display operation */
void display()
{
	int i;

	if(top == -1)
	{
	printf("Stack is empty\n");
	return;
	}

	printf("Stack elements:\n");
	for(i = top; i >= 0; i--)
	{
	printf("%d\n", stack[i]);
	}
}
```



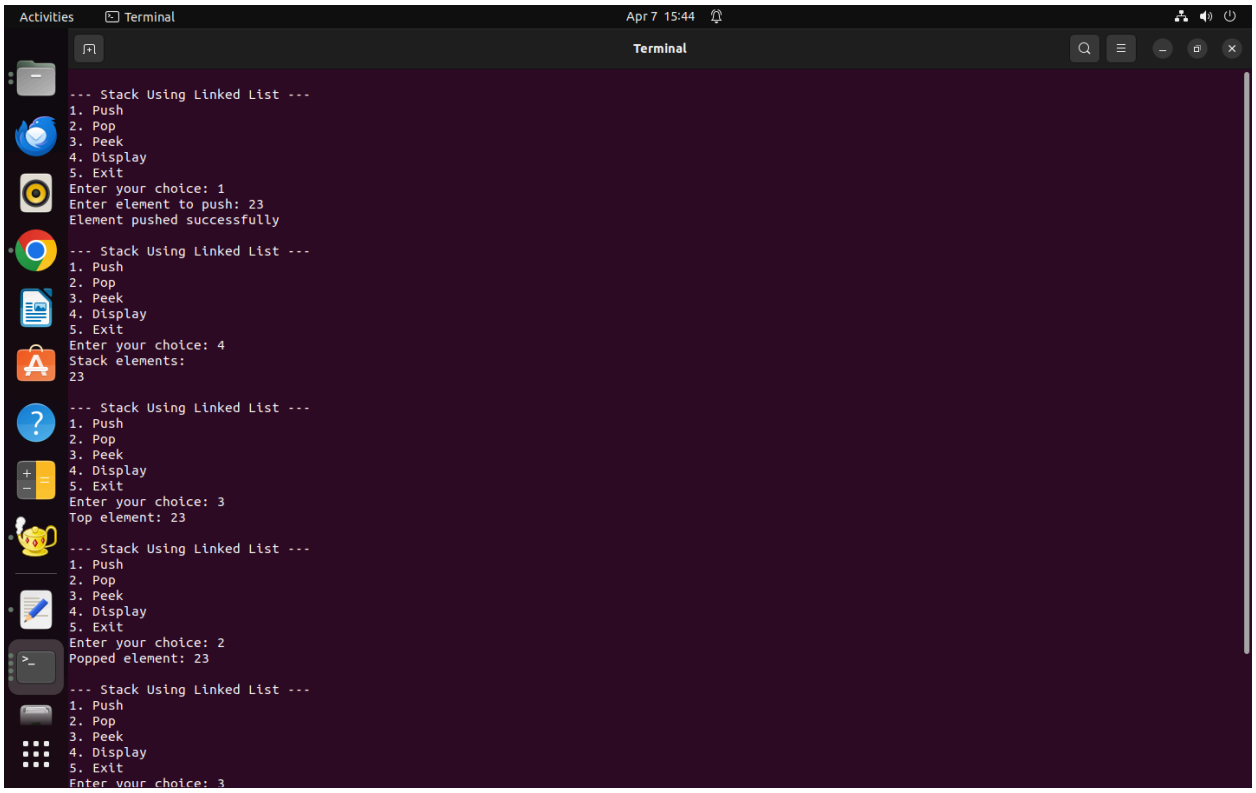
ii. Linked List

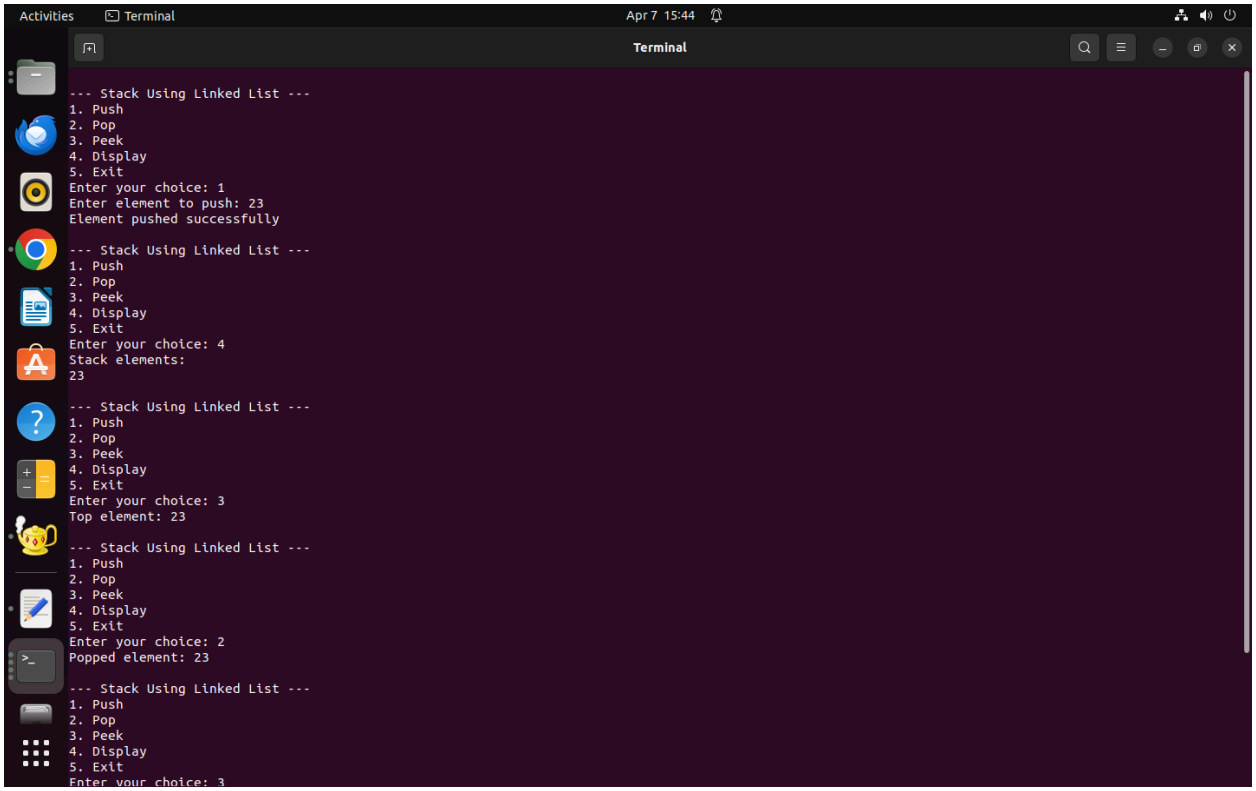
```
#include <stdio.h>
#include <stdlib.h>

struct node
{
        int data;
        struct node *next;
};

struct node *top = NULL;

/* Function declarations */
void push();
void pop();
void peek();
void display();

int main()
```

```
{
        int choice;

        while(1)
        {
        printf("\n--- Stack Using Linked List ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
        case 1: push(); break;
        case 2: pop(); break;
        case 3: peek(); break;
        case 4: display(); break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
        }
        }
}

/* Push operation */
void push()
{
        struct node *ptr;
        int value;

        ptr = (struct node*)malloc(sizeof(struct node));
        if(ptr == NULL)
        {
        printf("Stack Overflow\n");
        return;
        }

        printf("Enter element to push: ");
        scanf("%d", &value);

        ptr->data = value;
        ptr->next = top;
```

```
        top = ptr;

        printf("Element pushed successfully\n");
}

/* Pop operation */
void pop()
{
        struct node *temp;

        if(top == NULL)
        {
        printf("Stack Underflow\n");
        return;
        }

        temp = top;
        printf("Popped element: %d\n", temp->data);
        top = top->next;
        free(temp);
}

/* Peek operation */
void peek()
{
        if(top == NULL)
        {
        printf("Stack is empty\n");
        return;
        }

        printf("Top element: %d\n", top->data);
}

/* Display operation */
void display()
{
        struct node *temp;

        if(top == NULL)
        {
        printf("Stack is empty\n");
        return;
        }
```

```
        temp = top;
        printf("Stack elements:\n");
        while(temp != NULL)
        {
        printf("%d\n", temp->data);
        temp = temp->next;
        }
}
```





5. Write a program that implement Queue (its operations) using
1

i. Arrays ii. Linked List

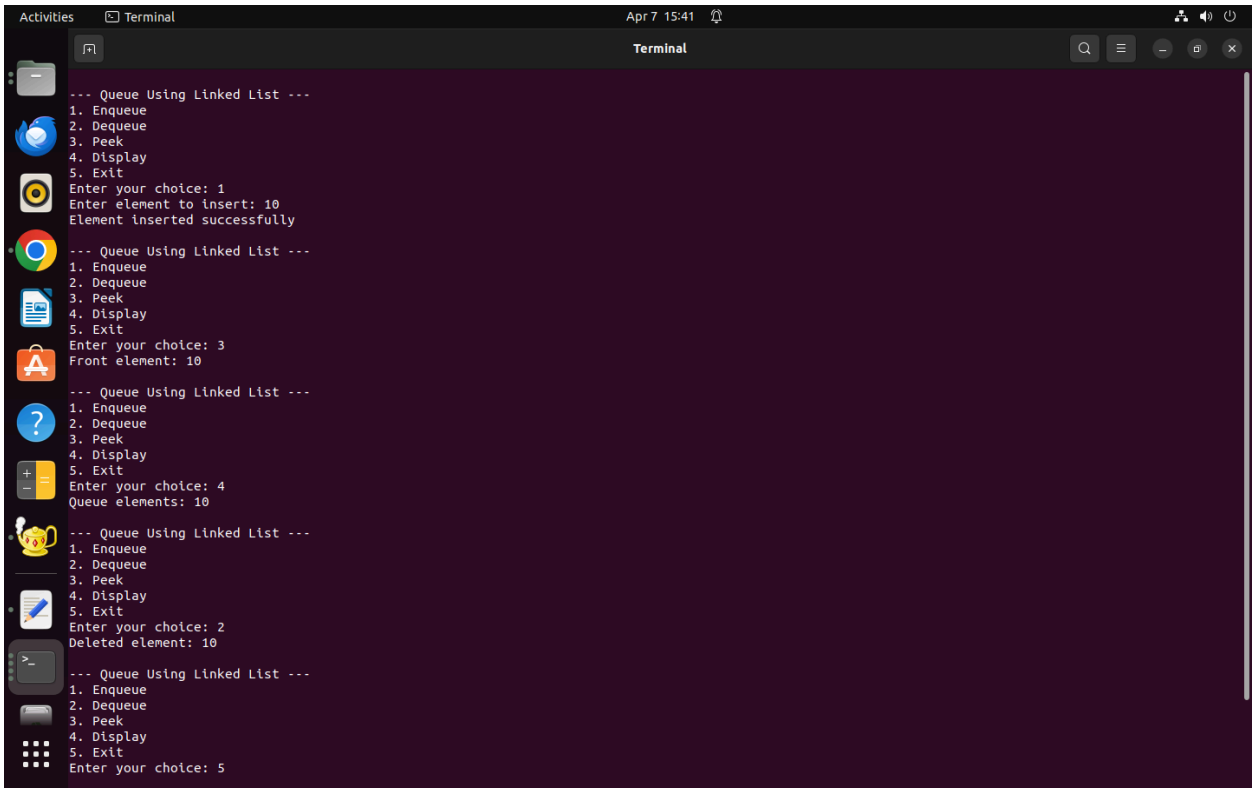
i. Arrays

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

/* Function declarations */
void enqueue();
void dequeue();
void peek();
void display();

int main()
```

```
{
        int choice;

        while(1)
        {
        printf("\n--- Queue Menu ---\n");
        printf("1. Enqueue(insert)\n");
        printf("2. Dequeue(delete)\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
        case 1: enqueue(); break;
        case 2: dequeue(); break;
        case 3: peek(); break;
        case 4: display(); break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
        }
        }
}

/* Enqueue operation */
void enqueue()
{
        int item;

        if(rear == MAX - 1)
        {
        printf("Queue Overflow\n");
        return;
        }

        printf("Enter element to insert: ");
        scanf("%d", &item);

        if(front == -1)
        front = 0;

        rear++;
```

```
	queue[rear] = item;

	printf("Element inserted successfully\n");
}

/* Dequeue operation */
void dequeue()
{
	if(front == -1 || front > rear)
	{
	printf("Queue Underflow\n");
	return;
	}

	printf("Deleted element: %d\n", queue[front]);
	front++;

	if(front > rear)
	{
	front = rear = -1;
	}
}

/* Peek operation */
void peek()
{
	if(front == -1)
	{
	printf("Queue is empty\n");
	return;
	}

	printf("Front element: %d\n", queue[front]);
}

/* Display operation */
void display()
{
	int i;

	if(front == -1)
	{
	printf("Queue is empty\n");
	return;
```

```
        }

        printf("Queue elements:\n");
        for(i = front; i <= rear; i++)
        {
        printf("%d ", queue[i]);
        }
        printf("/n")
}
```



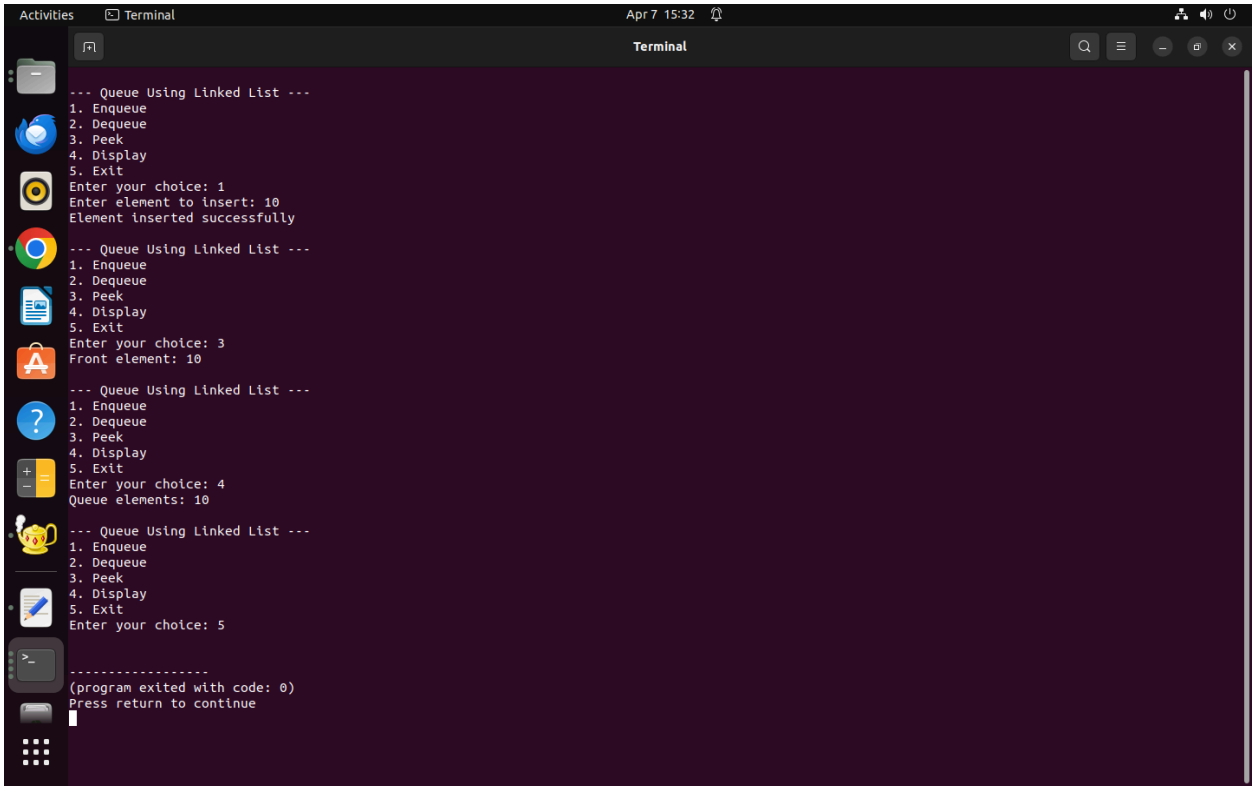
ii. Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
        int data;
        struct node *next;
};
```

```
struct node *front = NULL, *rear = NULL;

/* Function declarations */
void enqueue();
void dequeue();
void peek();
void display();

int main()
{
        int choice;

        while(1)
        {
        printf("\n--- Queue Using Linked List ---\n");
      printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
        case 1: enqueue(); break;
        case 2: dequeue(); break;
        case 3: peek(); break;
        case 4: display(); break;
        case 5: exit(0);
        default: printf("Invalid choice\n");
        }
        }
}

/* Enqueue operation */
void enqueue()
{
        struct node *ptr;
        int value;

        ptr = (struct node*)malloc(sizeof(struct node));
        if(ptr == NULL)
        {
```

```
        printf("Queue Overflow\n");
        return;
        }

        printf("Enter element to insert: ");
        scanf("%d", &value);

        ptr->data = value;
        ptr->next = NULL;

        if(rear == NULL)
        {
        front = rear = ptr;
        }
        else
        {
        rear->next = ptr;
        rear = ptr;
        }

        printf("Element inserted successfully\n");
}

/* Dequeue operation */
void dequeue()
{
        struct node *temp;

        if(front == NULL)
        {
        printf("Queue Underflow\n");
        return;
        }

        temp = front;
        printf("Deleted element: %d\n", temp->data);
        front = front->next;

        if(front == NULL)
        rear = NULL;

        free(temp);
}
```

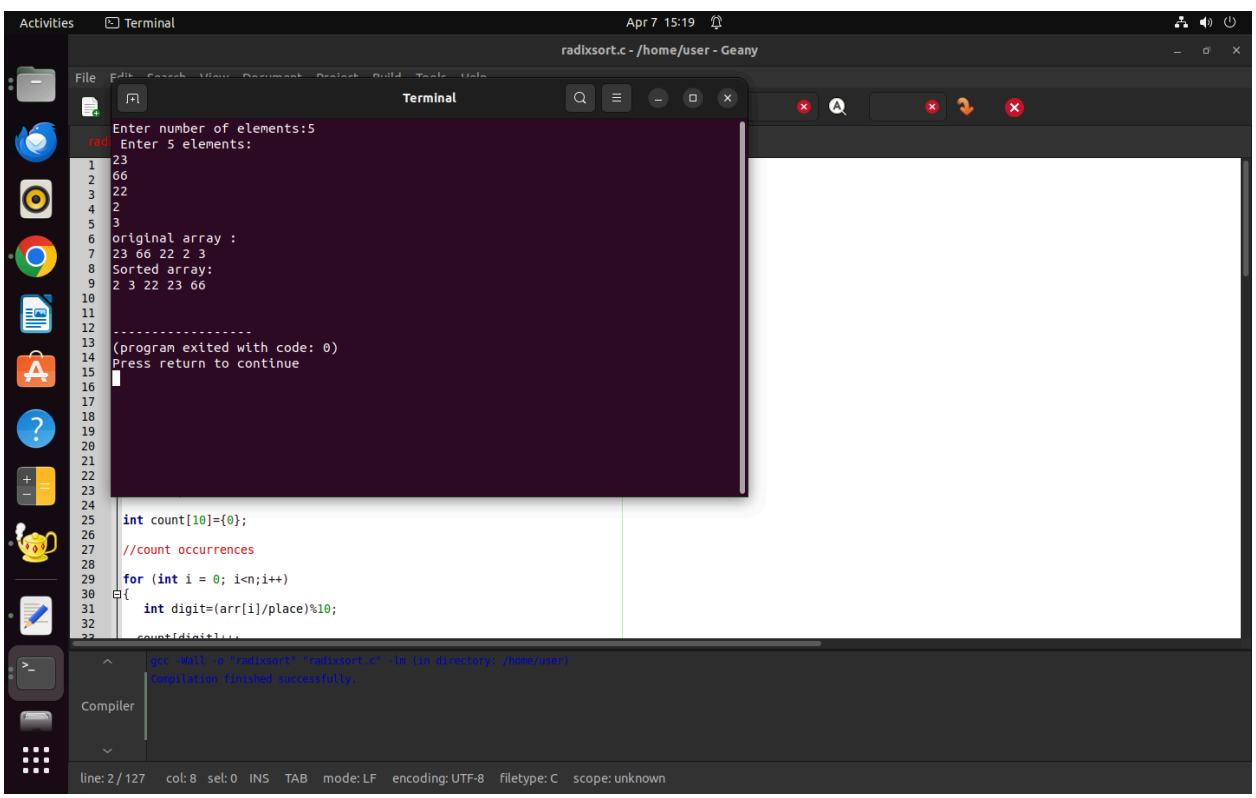
```
/* Peek operation */
void peek()
{
	if(front == NULL)
	{
	printf("Queue is empty\n");
	return;
	}

	printf("Front element: %d\n", front->data);
}

/* Display operation */
void display()
{
	struct node *temp;

	if(front == NULL)
	{
	printf("Queue is empty\n");
	return;
	}

	temp = front;
	printf("Queue elements: ");
	while(temp != NULL)
	{
	printf("%d ", temp->data);
	temp = temp->next;
	}
	printf("\n");
}
```



RADIX SORT

```
#include<stdio.h>
#include<stdlib.h>

//function to find maximum element

int getMax(int arr[],int n){

int max = arr[0];

for (int i=1; i<n; i++)
{

if(arr[i]>max)

max = arr[i];
}
return max;}

//Counting Sort based on digit
```

```
void CountingSort (int arr[], int n, int place) {

int *output = (int *)malloc(n*sizeof(int));

int count[10]={0};

//count occurrences

for (int i = 0; i<n;i++)
{
   int digit=(arr[i]/place)%10;

  count[digit]++;
}

// cummulative Count

for (int i=1;i<10;i++)
{
count[i]+= count[i-1];
}

//Build output array (stable)

for (int i=n-1; i>=0 ;i--) {

int digit = (arr[i]/place)%10;

output[count[digit]-1] = arr[i];

count[digit]--;
}

//copy back to original array

for (int i=0; i<n;i++){
arr[i]=output[i];
}

free(output); // free memory
}

//Radix Sort
```

```
void radixSort(int arr[] , int n) {

int Max= getMax(arr,n);

for (int place =1; Max/place >0; place*=10){

CountingSort(arr,n,place);

}

}

//print array

void printArray(int arr[], int n){

for (int i=0; i<n;i++) {

printf("%d ", arr[i]);
}

printf("\n");
}

// Main function

int main(){
int n;

printf("Enter number of elements:");

scanf("%d",&n);

// Dynamic allocation (no restriction)

int *arr= (int *)malloc(n* sizeof(int));

if(arr == NULL){
printf("Memory allocation failed\n");
return 1;
}

printf(" Enter %d elements:\n", n);
```

```
for (int i=0; i<n;i++) {
scanf("%d", &arr[i]);
}

printf("original array : \n");

printArray(arr, n);

radixSort(arr, n);

printf("Sorted array:\n");

printArray(arr, n);

free(arr);

return 0;
}
```